

Kubernetes Workloads

A Complete Teacher's Guide with Diagrams & Examples

Pods · ReplicaSets · Deployments · StatefulSets · DaemonSets · Jobs · HPA

Topic	Kubernetes Workload Resources
Audience	DevOps Students (Intermediate)
Duration	3–4 classroom hours
Version	Kubernetes 1.29+

What You Will Learn

- ◆ Full Kubernetes workload object hierarchy
- ◆ When to choose each workload type (Pod vs Deployment vs StatefulSet...)
- ◆ Zero-downtime rolling updates and rollbacks
- ◆ Resource requests/limits and Quality-of-Service classes
- ◆ HorizontalPodAutoscaler — automatic CPU/memory scaling
- ◆ Ready-to-use YAML manifests for every resource type

Table of Contents

Ch.	Topic	Key Concepts
1	Kubernetes Architecture Overview	Cluster · Node · Pod · Container hierarchy
2	Pods — The Atomic Unit	Spec · lifecycle · init/sidecar containers
3	ReplicaSets — Desired State	Self-healing · label selectors
4	Deployments — The Workhorse	Rolling updates · rollbacks · strategies
5	StatefulSets — Stateful Workloads	Ordered pods · stable DNS · per-pod PVCs
6	DaemonSets — Node-Level Pods	Logging · monitoring · one pod per node
7	Jobs & CronJobs — Batch Workloads	Run-to-completion · scheduled tasks
8	HorizontalPodAutoscaler	CPU/memory scaling · stabilization windows
9	Resource Management	Requests · limits · QoS classes · LimitRange
10	Quick Reference & Cheat Sheet	kubectl commands · comparison tables

1

Kubernetes Architecture Overview

Cluster → Node → Pod → Container

The Kubernetes Object Hierarchy

Every workload ultimately runs as one or more **Containers** inside a **Pod**, on a **Node**, inside a **Cluster**. Higher-level objects (Deployments, StatefulSets...) manage Pods — you rarely create Pods directly.

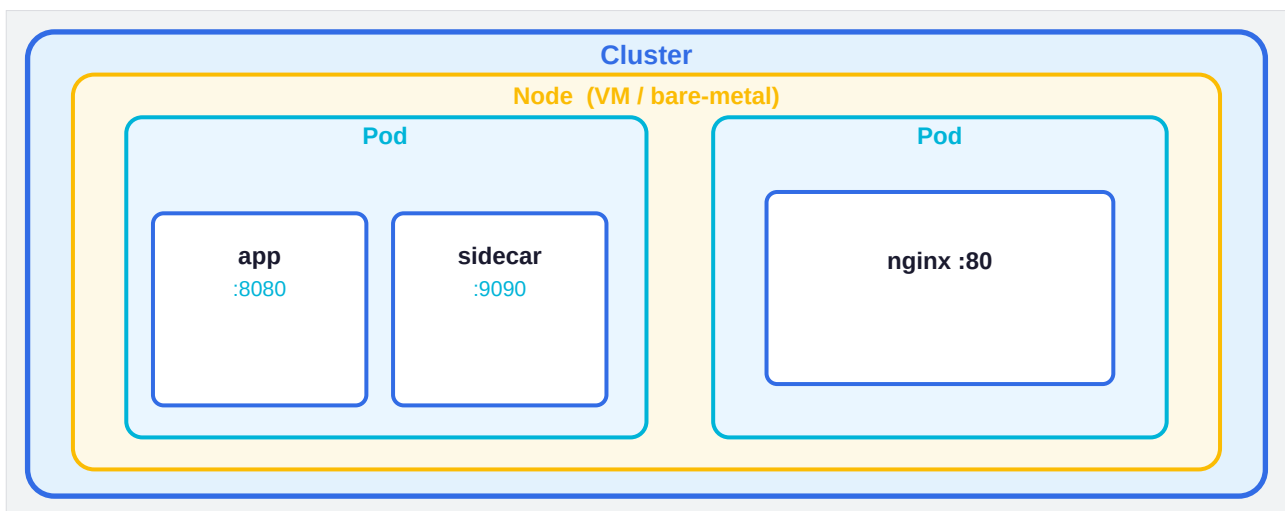


Figure 1.1 — Kubernetes object nesting: Cluster → Node → Pod → Container(s)

Object	Managed by	Key property
Container	Container runtime (containerd)	Runs the actual process
Pod	kubelet on each Node	Shares network + storage between containers
ReplicaSet	kube-controller-manager	Maintains N identical Pod copies
Deployment	kube-controller-manager	Adds rolling-update + rollback to ReplicaSet
StatefulSet	kube-controller-manager	Ordered pods with stable identity + PVCs
DaemonSet	kube-controller-manager	One pod per node
Job / CronJob	kube-controller-manager	Run-to-completion / scheduled batch

2

Pods — The Atomic Unit

Spec · lifecycle · init containers · sidecar pattern

A **Pod** wraps one or more containers that share the same network namespace (*localhost*) and optionally the same storage volumes. Pods are **ephemeral** — they are not rescheduled if a node dies; a controller must do that.

Pod Lifecycle

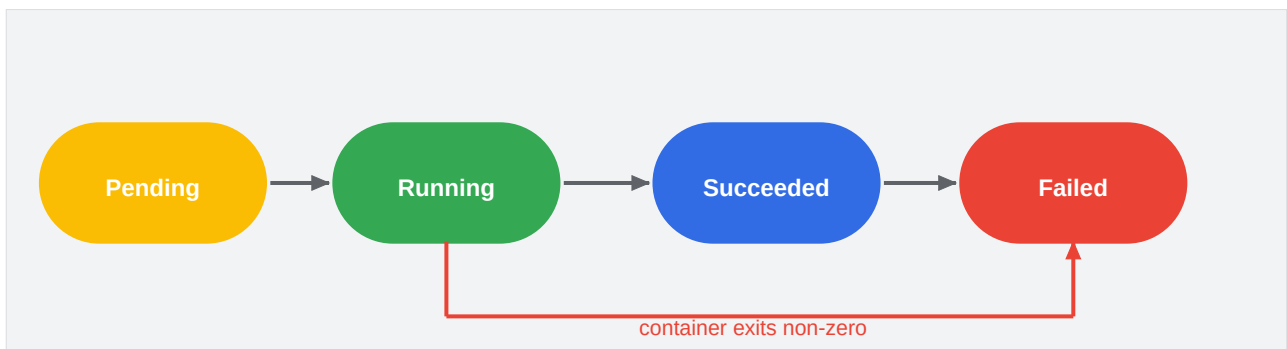


Figure 2.1 — Pod state machine. Liveness probe failure → restart. Readiness probe failure → removed from Service endpoints.

Minimal Pod YAML

```
pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-app
  labels:
    app: my-app
spec:
  containers:
  - name: app
    image: nginx:1.27-alpine
    ports:
      - containerPort: 80
    resources:
      requests:
        cpu: 50m
        memory: 64Mi
      limits:
        cpu: 250m
        memory: 128Mi
    readinessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 5
      periodSeconds: 10
    livenessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 15
      periodSeconds: 20
```

InitContainer Pattern

initContainers run sequentially and must all succeed before any regular container starts. Perfect for: waiting for dependencies, running DB migrations, fetching secrets.

init-container.yaml

spec:

initContainers:

- name: wait-for-db

image: busybox:1.36

command: [sh,-c,"until nc -z postgres 5432; do sleep 2; done"]

containers:

- name: api

image: my-api:v2

■ **Classroom Question:** What happens if an initContainer fails? (Pod restarts per *restartPolicy*, default: Always. The main containers NEVER start.)

3

ReplicaSets — Desired State

Self-healing · label selectors · replica management

A **ReplicaSet** continuously reconciles the *desired* number of Pod replicas with the *actual* count. If a Pod is deleted or crashes, the RS creates a replacement. If extras appear with matching labels, the RS deletes them.

```
replicaset.yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: web-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web      # watches ALL pods with this label
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.27-alpine
          ports:
            - containerPort: 80
```

■ ■ **Teaching Note:** In practice you never write ReplicaSet YAMLs directly. Always use a **Deployment** — it manages ReplicaSets for you and adds rolling-update and rollback support.

4

Deployments — The Workhorse

Rolling updates · rollbacks · update strategies

A **Deployment** is the most-used workload resource. It wraps a ReplicaSet and adds **declarative update semantics**: change the image tag in the YAML, apply it, and Kubernetes figures out how to transition safely.

Rolling Update — Zero Downtime

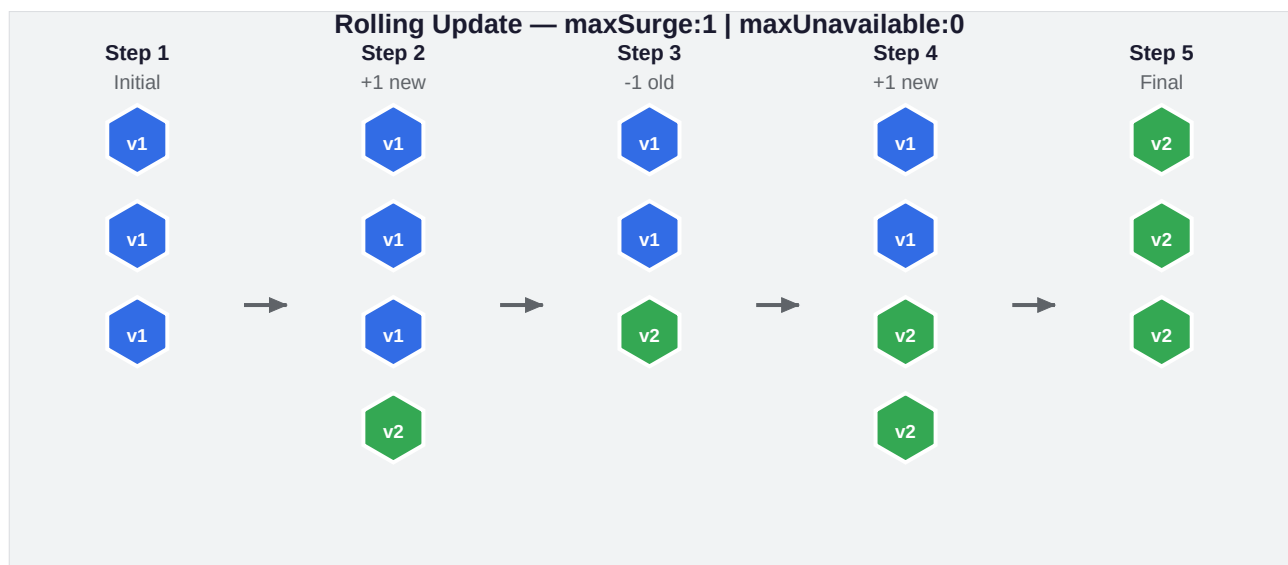


Figure 4.1 — Rolling update v1 → v2. With `maxUnavailable:0`, traffic is always served by healthy pods.

```
deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1      # allow 1 extra pod above desired
      maxUnavailable: 0 # no pods go unavailable (zero-downtime)
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.27-alpine # change tag to trigger rollout
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: 50m
              memory: 64Mi
            limits:
              cpu: 500m
              memory: 128Mi
```

Key Commands

Command	Effect
<code>kubectl apply -f deployment.yaml</code>	Create or update deployment
<code>kubectl set image deploy/web-deploy web=nginx:1.28</code>	Trigger rolling update
<code>kubectl rollout status deploy/web-deploy</code>	Watch rollout progress live
<code>kubectl rollout undo deploy/web-deploy</code>	Rollback to previous version
<code>kubectl rollout history deploy/web-deploy</code>	View change history
<code>kubectl scale deploy/web-deploy --replicas=5</code>	Manual scaling

■ **Lab Exercise:** Deploy nginx:1.26, then update to nginx:1.27. Watch the rollout. Then deploy nginx:BROKEN to trigger a failure and practice *kubectl rollout undo*.

5

StatefulSets — Stateful Workloads

Ordered pods · stable hostname · dedicated PVC per pod

Use a **StatefulSet** when your application needs: (1) a stable network identity (*pod-0.headless-svc*), (2) its own persistent volume per replica, (3) ordered, graceful startup and shutdown.

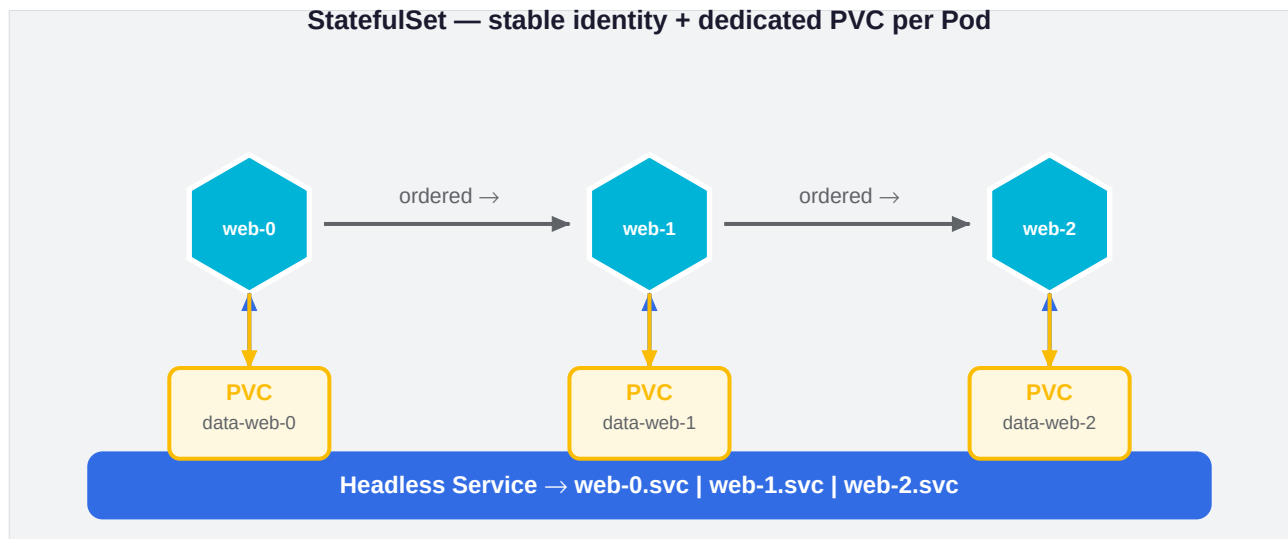


Figure 5.1 — StatefulSet: each Pod gets a stable hostname and a dedicated PVC that outlives the Pod.

```
statefulset.yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: web-headless # must match a Headless Service
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.27-alpine
          volumeMounts:
            - name: data
              mountPath: /data
  volumeClaimTemplates: # 1 PVC per Pod, created automatically
    - metadata:
        name: data
      spec:
        accessModes: [ReadWriteOnce]
        resources:
          requests:
            storage: 1Gi
```

Feature	Deployment	StatefulSet
Pod names	Random (deploy-abc)	Stable (web-0, web-1, web-2)
Startup order	Parallel	Sequential: 0 then 1 then 2
DNS hostname	Not stable	web-0.web-headless.ns.svc
Per-pod storage	Shared or none	Dedicated PVC per Pod
Use cases	APIs, frontends	Postgres, Redis, Kafka, Zookeeper

6

DaemonSets — Node-Level Pods

One Pod per Node · auto-scheduled on new nodes

A **DaemonSet** guarantees that one copy of a Pod runs on **every Node** (or a subset via nodeSelector/tolerations). When a new node joins the cluster the Pod is automatically scheduled onto it.

DaemonSet — exactly one Pod per Node (auto on new nodes)

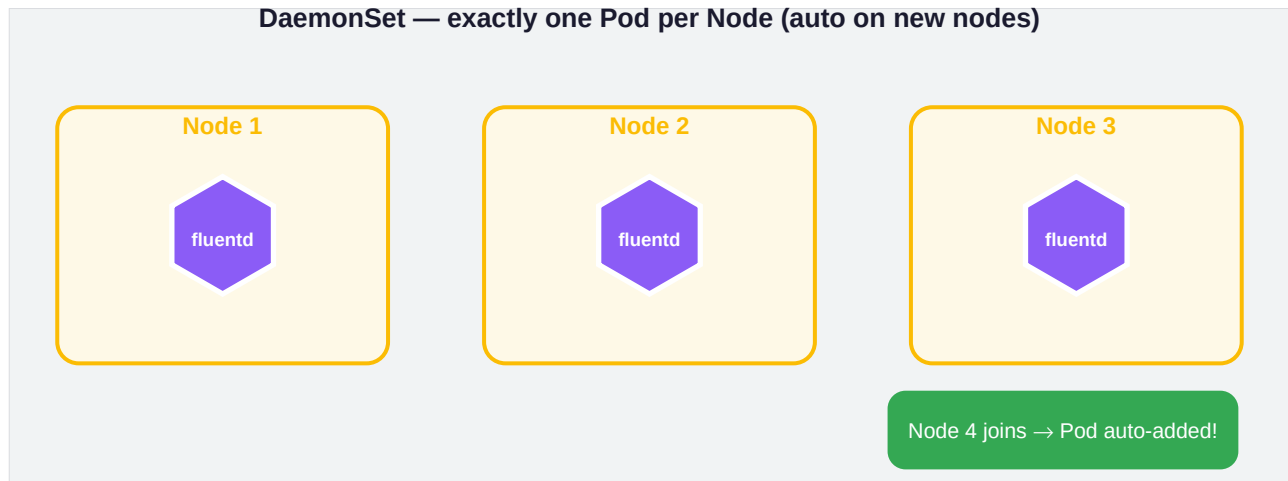


Figure 6.1 — DaemonSet places one Pod per Node; new nodes receive the Pod automatically.

Use Case	Example Tools
Node-level log collection	Fluentd, Fluent Bit, Filebeat
Node monitoring agent	Prometheus node-exporter, Datadog Agent
CNI network plugin	Calico, Cilium, Flannel
Distributed storage daemon	Rook/Ceph, GlusterFS
Security / runtime scanning	Falco, Sysdig, Tetragon

```
daemonset.yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-logger
  namespace: kube-system
spec:
  selector:
    matchLabels:
      app: node-logger
  template:
    metadata:
      labels:
        app: node-logger
    spec:
      tolerations:
        - key: node-role.kubernetes.io/control-plane
          effect: NoSchedule
      containers:
        - name: fluent-bit
          image: fluent/fluent-bit:3.0
          resources:
            limits:
              memory: 128Mi
              cpu: 100m
          volumeMounts:
            - name: varlog
              mountPath: /var/log
      volumes:
        - name: varlog
          hostPath:
            path: /var/log
```

7

Jobs & CronJobs — Batch Workloads

Run-to-completion · automatic retries · scheduled tasks

Jobs create one or more Pods and track them until completion. Completed Pods are *not* restarted. Failed Pods are retried up to *backoffLimit* times.

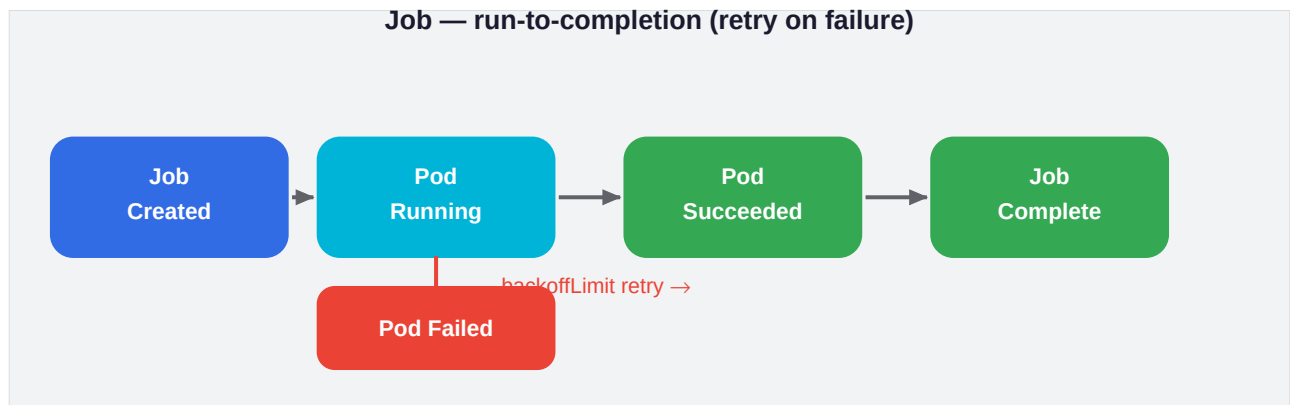


Figure 7.1 — Job lifecycle: Pod failure triggers retry; success marks Job Complete.

```
job.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migration
spec:
  completions: 1      # total successful completions required
  parallelism: 1      # pods running in parallel
  backoffLimit: 3     # max retries on failure
  template:
    spec:
      restartPolicy: OnFailure # Never or OnFailure (not Always)
      containers:
        - name: migrate
          image: my-app:v2
          command: [python, manage.py, migrate]
```

CronJobs — Scheduled Jobs

A **CronJob** creates a new Job on a cron schedule. It uses standard Unix cron syntax.

```
cronjob.yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: daily-report
spec:
  schedule: "0 2 * * *"      # every day at 02:00 UTC
  concurrencyPolicy: Forbid  # skip new Job if previous still running
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: report
              image: my-reporter:v1
              command: [python, report.py]
```

■ **Lab Idea:** Create a CronJob that runs every minute (* * * * *), watch pods appear with `kubectrl get pods -w`, then inspect completed pods with `kubectrl logs`.

8

HorizontalPodAutoscaler

CPU & memory scaling · min/max replicas · stabilization

The **HPA** automatically adjusts the number of Pod replicas based on observed metrics (CPU, memory, or custom). It reads data from the **Metrics Server** every 15 seconds and calculates:

$$\text{desiredReplicas} = \text{ceil}(\text{currentReplicas} \times \text{currentMetric} / \text{targetMetric})$$

HorizontalPodAutoscaler — CPU-driven scale-up / scale-down

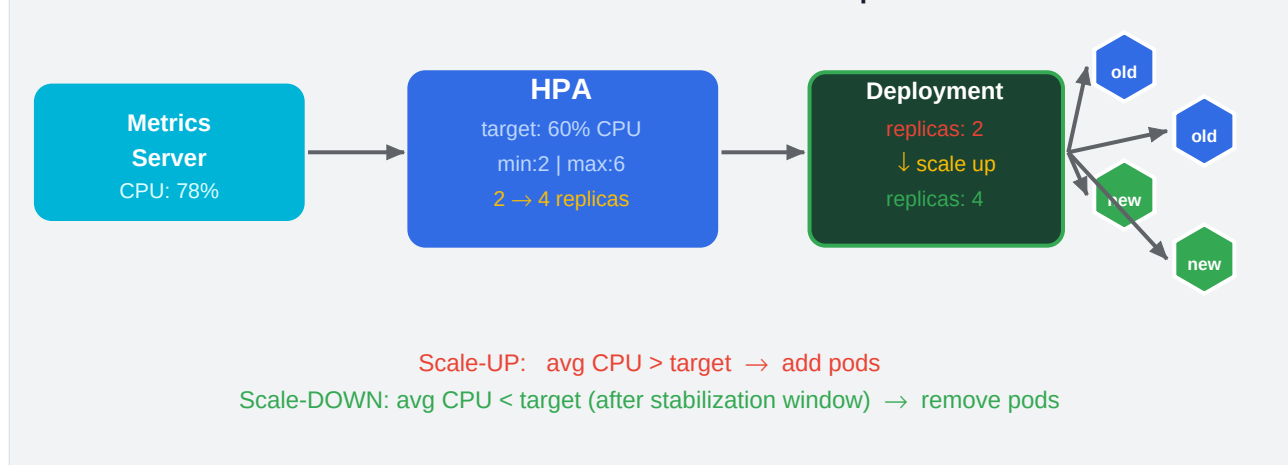


Figure 8.1 — HPA reads CPU from Metrics Server and scales the Deployment.

```
hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-deploy
  minReplicas: 2
  maxReplicas: 8
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60    # scale up when avg CPU > 60%
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 75
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60    # wait 1 min before scaling up
    scaleDown:
      stabilizationWindowSeconds: 300    # wait 5 min before scaling down
```

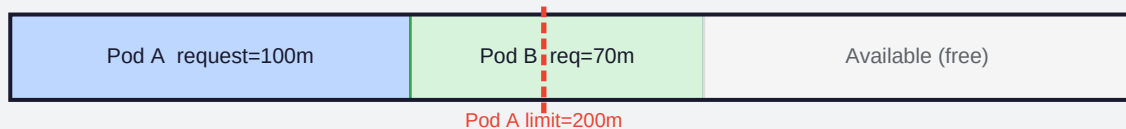
■ ■ **Prerequisite:** HPA requires the **Metrics Server** to be installed. On minikube: *minikube addons enable metrics-server*. Without it, HPA shows for current CPU.

9

Resource Management

Requests · Limits · QoS Classes · LimitRange

Resource Requests vs Limits — Node CPU allocation



Scheduler uses REQUESTS to decide placement.

Containers are THROTTLED at the cpu limit and OOMKilled at the memory limit.

- Request = guaranteed minimum
- Limit = hard ceiling (throttle/OOMKill)

Figure 9.1 — Requests are used for scheduling; limits are enforced at runtime.

Requests vs Limits

	Request	Limit
CPU	Guaranteed minimum. Used by scheduler to find a node with enough free CPU.	Hard ceiling. Container is CPU-throttled if it tries to exceed this.
Memory	Guaranteed minimum. Scheduler uses this for placement.	Hard ceiling. Container is OOMKilled if it exceeds this.

QoS Classes

QoS Class	Condition	Eviction Priority (low = evicted first)
BestEffort	No requests or limits set	Evicted first under pressure
Burstable	Requests set, limits > requests (or no limits)	Middle priority
Guaranteed	Requests == Limits for ALL containers	Last to be evicted

LimitRange — Cluster Defaults

A **LimitRange** sets default requests/limits for a namespace so Pods without explicit resources still get sensible values.

```
limitrange.yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: default
spec:
  limits:
    - type: Container
      default:           # applied as limit if none specified
        cpu: 500m
        memory: 128Mi
      defaultRequest:    # applied as request if none specified
        cpu: 100m
        memory: 64Mi
      max:               # pods cannot exceed these
        cpu: 2
        memory: 1Gi
```

10

Quick Reference & Cheat Sheet

kubectl commands · workload comparison

Workload Comparison at a Glance

Workload	Replicas?	Ordered?	Stable ID?	Persistent Storage?	Use when...
Pod	No	—	No	Shared only	Quick testing only
ReplicaSet	Yes	No	No	Shared only	(use Deployment instead)
Deployment	Yes	No	No	Shared only	Stateless services, APIs, frontends
StatefulSet	Yes	Yes	Yes	Per-pod PVC	Databases, message queues
DaemonSet	1/node	No	No	HostPath/shared	Logging, monitoring agents
Job	Configurable	No	No	Shared only	Batch tasks, DB migrations
CronJob	Configurable	No	No	Shared only	Scheduled batch tasks

Essential kubectl Commands

Command	Description
<code>kubectl get pods -n <ns> -w</code>	Watch pods live
<code>kubectl describe pod <name></code>	Show events, probes, resource limits
<code>kubectl logs -f deploy/<name></code>	Stream logs from deployment
<code>kubectl exec -it <pod> -- sh</code>	Shell into a pod
<code>kubectl apply -f file.yaml</code>	Create or update any resource
<code>kubectl delete -f file.yaml</code>	Delete resources from file
<code>kubectl rollout status deploy/<name></code>	Monitor a rolling update
<code>kubectl rollout undo deploy/<name></code>	Rollback one revision
<code>kubectl scale deploy/<name> --replicas=4</code>	Manual scale
<code>kubectl top pods</code>	Show CPU/memory usage (needs Metrics Server)
<code>kubectl get events --sort-by=.lastTimestamp</code>	Debug cluster events

■ **Final Classroom Challenge:** Students deploy the full Job Board lab (from the DevSecOps project) to a minikube cluster using only manifests from the k8s/ directory — no docker-compose allowed!